



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



APPLIED ALGORITHMS



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

APPLIED ALGORITHMS

Geometric algorithm

ONE LOVE. ONE FUTURE.

CONTENTS

- Basic formular
- Find the convex hull (bao lồi)
- Check if a point is inside a convex polygon (đa giác lồi)

Basic formular

- Point

```
struct Point {  
    double x, y;  
};
```

- Straight line $ax + by + c = 0$

```
struct Line {  
    double a, b, c;  
};
```

- Vector $\overrightarrow{AB}$ of two points $A(x_A, y_A)$ and $B(x_B, y_B)$
$$\overrightarrow{AB} = (x_B - x_A, y_B - y_A)$$

Basic formular

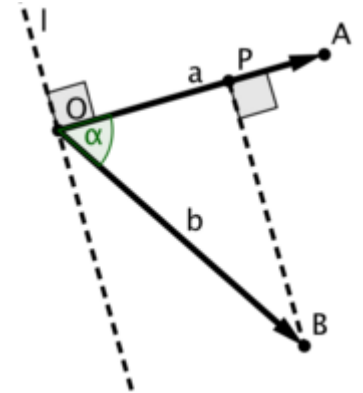
3 points $A(x_A, y_A)$, $B(x_B, y_B)$ and $C(x_C, y_C)$ are collinear when:

- $\overrightarrow{AB} = k \times \overrightarrow{AC}$
- $x_B - x_A = k \times (x_C - x_A)$
- $y_B - y_A = k \times (y_C - y_A)$

To avoid division by zero: $(x_A - x_B) \times (y_A - y_C) = (x_A - x_C) \times (y_A - y_B)$

Basic formular

- Scalar product of $\overrightarrow{OA}(x_a, y_a)$ and $\overrightarrow{OB}(x_b, y_b)$
- $\overrightarrow{OA} \cdot \overrightarrow{OB} = x_a x_b + y_a y_b = |\overrightarrow{OA}| |\overrightarrow{OB}| \cos \alpha = \sqrt{x_a^2 + y_a^2} \sqrt{x_b^2 + y_b^2} \cos \alpha,$
- $\cos \alpha = \frac{x_a x_b + y_a y_b}{\sqrt{x_a^2 + y_a^2} \sqrt{x_b^2 + y_b^2}}$
- Drawing the line d perpendicular to $\overrightarrow{OA}$, based on $\cos \alpha$ we have:
 - If $\overrightarrow{OA} \cdot \overrightarrow{OB} > 0$ then A and B are on the same side of line d
 - If $\overrightarrow{OA} \cdot \overrightarrow{OB} = 0$ then B lies on line d
 - If $\overrightarrow{OA} \cdot \overrightarrow{OB} < 0$ then A and B are on different sides of line d



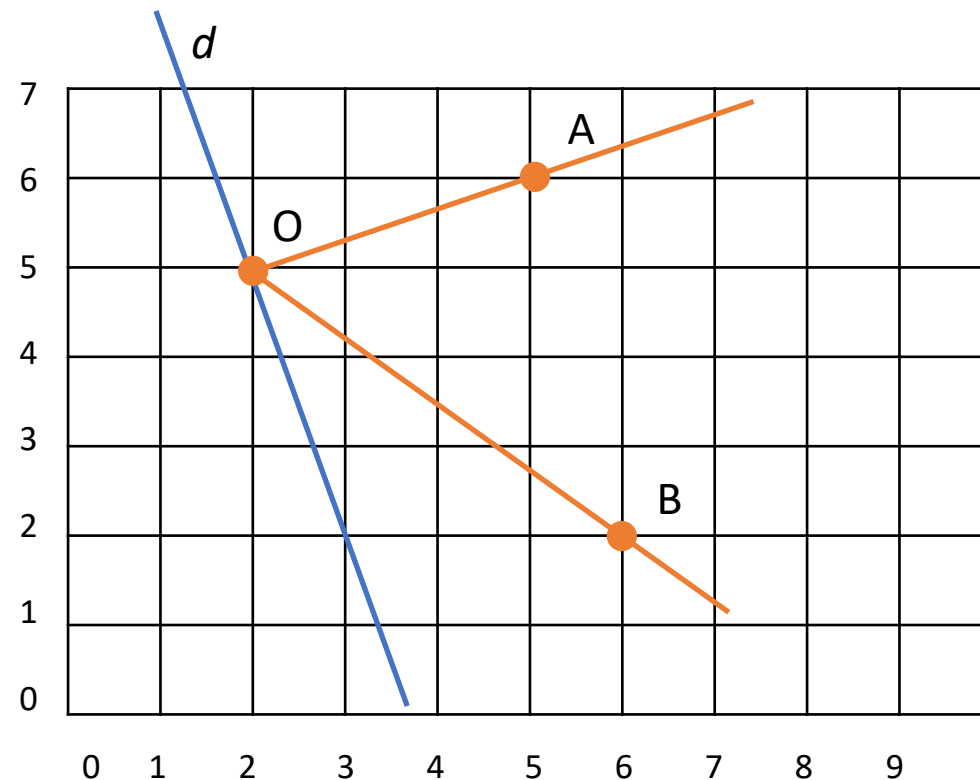
Basic formular

```
struct Point {
    int x, y;
    Point(int x, int y) : x(x), y(y) {}
};

double dist(Point &a, Point &b) {
    long long x = a.x - b.x; long long y = a.y - b.y;
    return sqrt(1LL * x*x + 1LL * y*y);
}

long long dot_product(Point &O, Point &A, Point &B) {
    long long xa = A.x - O.x; long long ya = A.y - O.y;
    long long xb = B.x - O.x; long long yb = B.y - O.y;
    return 1LL * xa * xb + 1LL * ya * yb;
}

int main(){
    Point O(2,5); Point A(5,6); Point B(6,2);
    double cos = dot_product(O,A,B)*1.0/(dist(O,A)*dist(O,B));
    cout << "cos = " << cos << endl;
}
```



cos = 0.56921

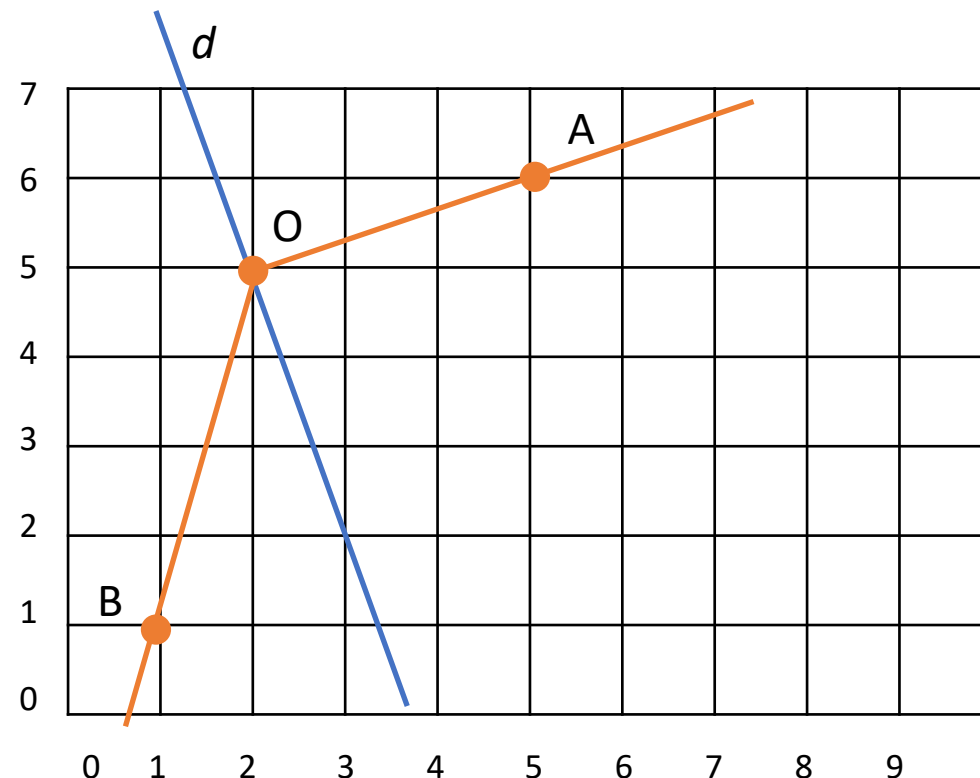
Basic formular

```
struct Point {
    int x, y;
    Point(int x, int y) : x(x), y(y) {}
};

double dist(Point &a, Point &b) {
    long long x = a.x - b.x; long long y = a.y - b.y;
    return sqrt(1LL * x*x + 1LL * y*y);
}

long long dot_product(Point &O, Point &A, Point &B) {
    long long xa = A.x - O.x; long long ya = A.y - O.y;
    long long xb = B.x - O.x; long long yb = B.y - O.y;
    return 1LL * xa * xb + 1LL * ya * yb;
}

int main(){
    Point O(2,5); Point A(5,6); Point B(1,1);
    double cos = dot_product(O,A,B)*1.0/(dist(O,A)*dist(O,B));
    cout << "cos = " << cos << endl;
}
```

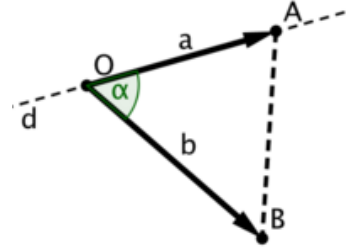


cos = -0.536875

Basic formular

- The direct product of $\overrightarrow{OA}(x_a, y_a)$ and $\overrightarrow{OB}(x_b, y_b)$
- $\overrightarrow{OA} \times \overrightarrow{OB} = x_a y_b - y_a x_b = |\overrightarrow{OA}| |\overrightarrow{OB}| \sin \alpha$

$$\overrightarrow{OA} \times \overrightarrow{OB} = x_a y_b - y_a x_b = \sqrt{x_a^2 + y_a^2} \sqrt{x_b^2 + y_b^2} \sin \alpha, \quad \sin \alpha = \frac{x_a y_b - y_a x_b}{\sqrt{x_a^2 + y_a^2} \sqrt{x_b^2 + y_b^2}}$$



- Draw line d to coincide with $\overrightarrow{OA}$, based on $\sin \alpha$ we have
 - If $\overrightarrow{OA} \times \overrightarrow{OB} > 0$ then B is on the left of line d (the direction of rotation from ray $\overrightarrow{OA}$ to ray $\overrightarrow{OB}$ is counterclockwise)
 - If $\overrightarrow{OA} \times \overrightarrow{OB} = 0$ then B belongs to the line d
 - If $\overrightarrow{OA} \times \overrightarrow{OB} < 0$ then B is on the right of line d (the direction of rotation from ray $\overrightarrow{OA}$ to ray $\overrightarrow{OB}$ is clockwise)
- The absolute value of the direct product of two vectors $\overrightarrow{OA}$ và $\overrightarrow{OB}$ is equal to twice the area of triangle OAB .

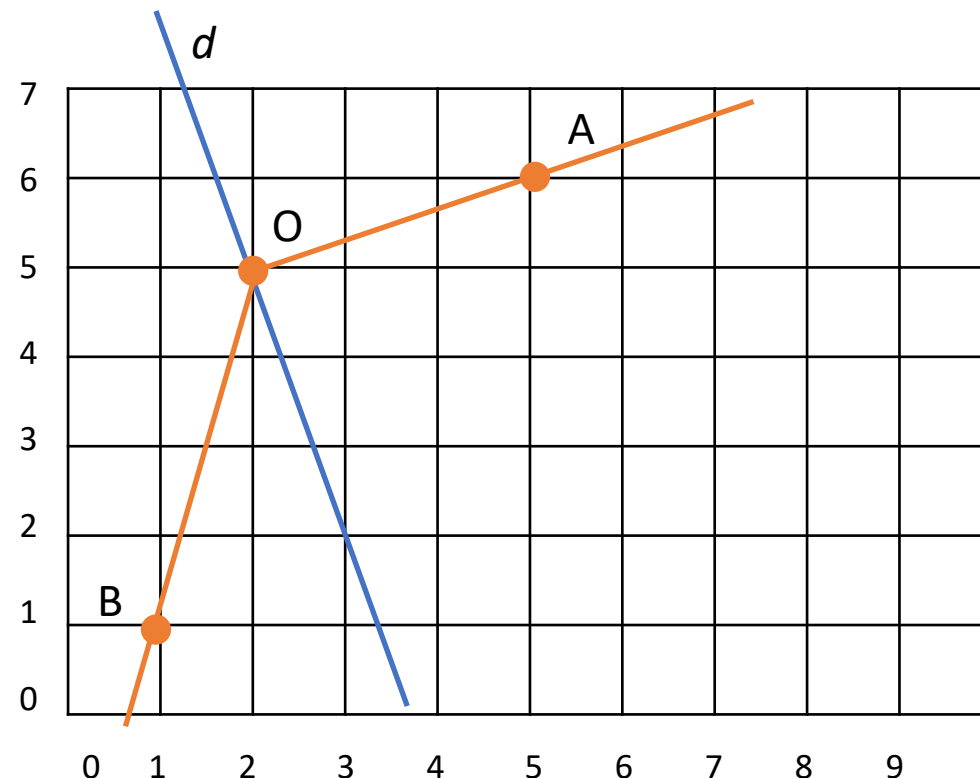
Basic formular

```
struct Point {
    int x, y;
    Point(int x, int y) : x(x), y(y) {}
};

double dist(Point &a, Point &b) {
    long long x = a.x - b.x; long long y = a.y - b.y;
    return sqrt(1LL * x*x + 1LL * y*y);
}

long long cross_product(Point &O, Point &A, Point &B) {
    //scalar product of 2 vectors (O,A).(O,B)
    long long xa = A.x - O.x; long long ya = A.y - O.y;
    long long xb = B.x - O.x; long long yb = B.y - O.y;
    return 1LL * xa * yb - 1LL * ya * xb;
}

int main(){
    Point O(2,5); Point A(5,6); Point B(1,1);
    double sin = cross_product(O,A,B)*1.0/(dist(O,A)*dist(O,B));
    cout << "sin = " << sin << endl;
}
```



sin = -0.843661

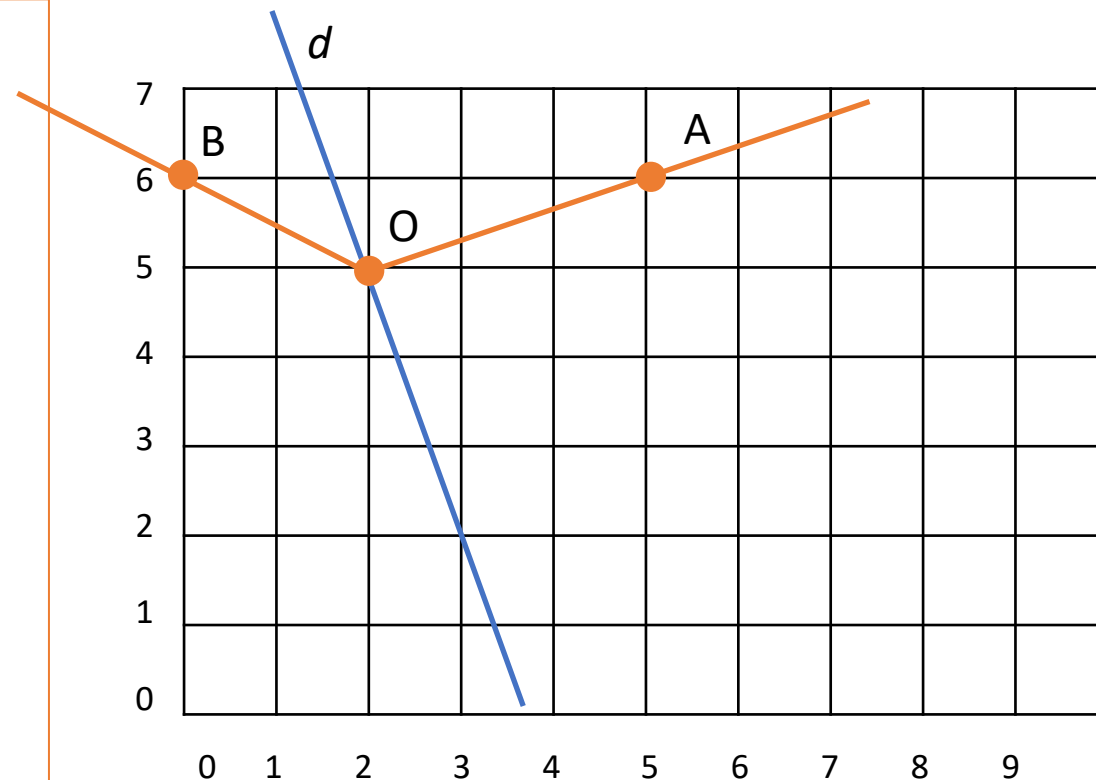
Basic formular

```
struct Point {
    int x, y;
    Point(int x, int y) : x(x), y(y) {}
};

double dist(Point &a, Point &b) {
    long long x = a.x - b.x; long long y = a.y - b.y;
    return sqrt(1LL * x*x + 1LL * y*y);
}

long long cross_product(Point &O, Point &A, Point &B) {
    //scalar product of 2 vectors (O,A).(O,B)
    long long xa = A.x - O.x; long long ya = A.y - O.y;
    long long xb = B.x - O.x; long long yb = B.y - O.y;
    return 1LL * xa * yb - 1LL * ya * xb;
}

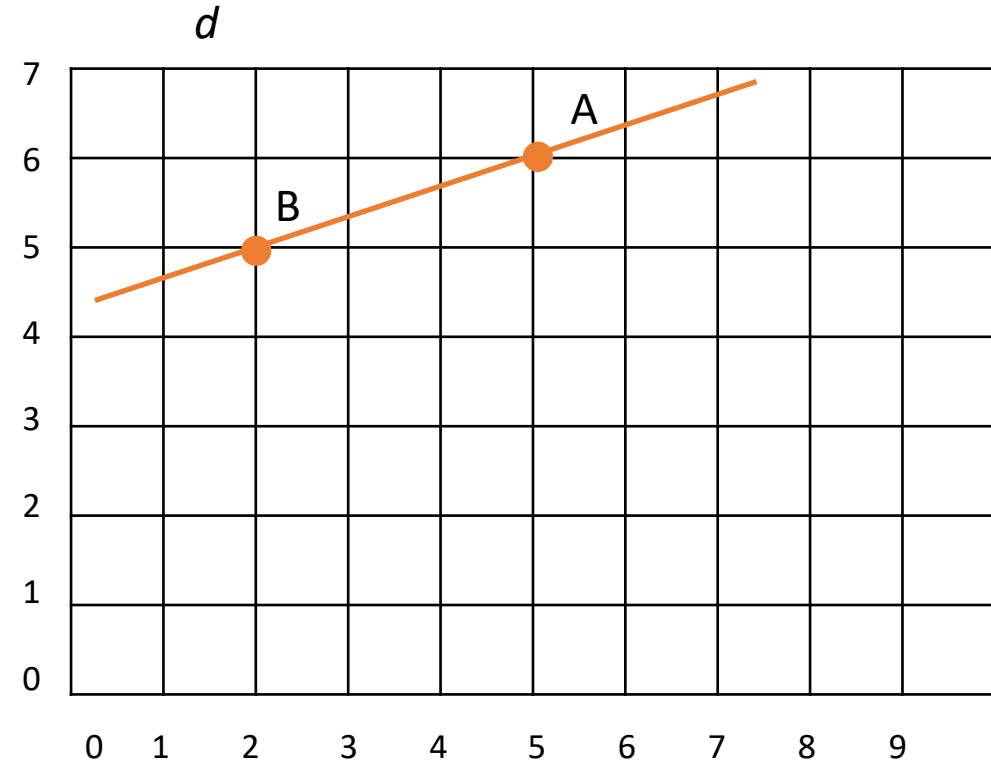
int main(){
    Point O(2,5); Point A(5,6); Point B(0,6);
    double sin = cross_product(O,A,B)*1.0/(dist(O,A)*dist(O,B));
    cout << "sin = " << sin << endl;
}
```



sin = 0.707107

Basic formular

- Given the straight line l_{AB} passing through two points $A(x_A, y_A)$ and $B(x_B, y_B)$
 - Vector $\overrightarrow{AB} = (x_B - x_A, y_B - y_A)$
 - Perpendicular vector $\vec{v} = (y_A - y_B, x_B - x_A)$
 - For every point $P(x, y)$ lying on the line l_{AB} , then $\overrightarrow{AP} \cdot \vec{v} = 0$
$$(x - x_A)(y_A - y_B) + (y - y_A)(x_B - x_A) = 0$$
 - The equation of the line l_{AB} là $ax + by + c = 0$
$$a = y_A - y_B$$
$$b = x_B - x_A$$
$$c = x_A y_B - y_A x_B$$
- Example: the straight line passing through $A(5, 6)$ and $B(2, 5)$ has equation: $(6-5)x + (2-5)y + (5 \times 5 - 6 \times 2) = 0$
hay $x - 3y + 13 = 0$



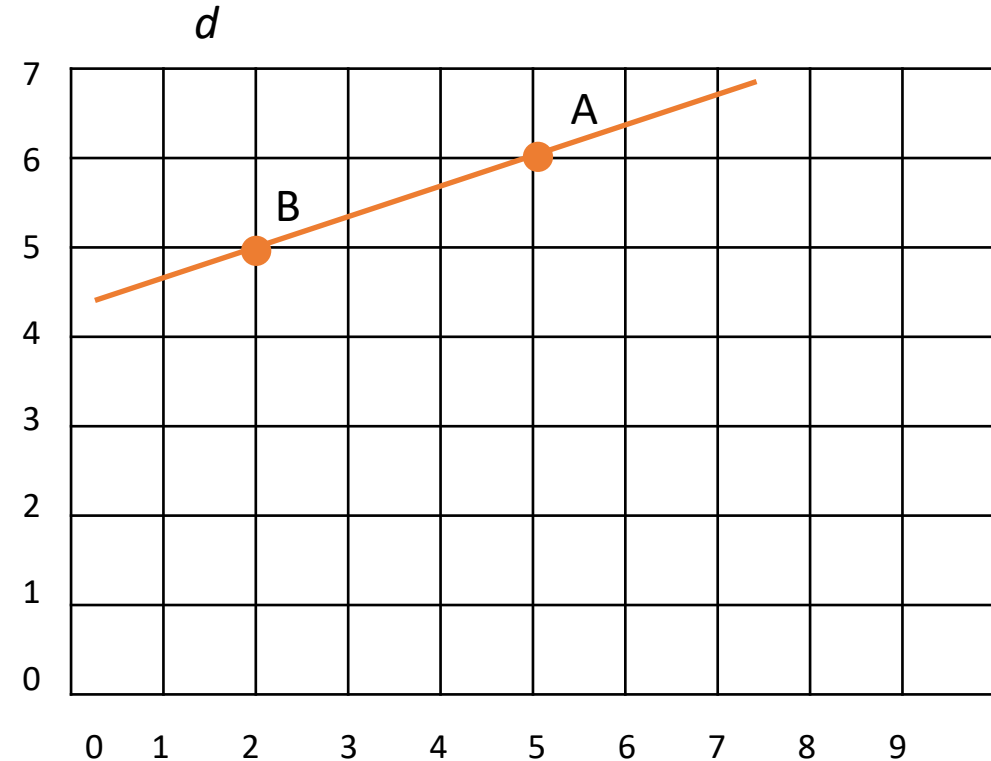
Basic formular

```
struct Point {
    int x, y;
    Point(int x, int y) : x(x), y(y) {}
};

struct Line{
    int a,b,c;
};

void makeLine(Point& A, Point& B, Line& L){
    L.a = A.y - B.y;
    L.b = B.x - A.x;
    L.c = A.x*B.y - A.y*B.x;
}

int main(){
    Point A(5,6); Point B(2,5); Line L;
    makeLine(A,B,L);
    cout << L.a << "x" << " + " << L.b << "y + " << L.c << " = 0";
}
```



Basic formular

- Straight line l_{AB} passing through 2 points $A(x_A, y_A)$ and $B(x_B, y_B)$

- $(y_A - y_B)x + (x_B - x_A)y + (x_A y_B - y_A x_B) = 0$ (1)

- The distance from point $C(x_C, y_C)$ to line l_{AB} is d

- The area of triangle ABC is:

- $S_{ABC} = \frac{|\overrightarrow{AB} \times \overrightarrow{AC}|}{2} = \frac{|\overrightarrow{AB}| \times d}{2} \Rightarrow d = \frac{|\overrightarrow{AB} \times \overrightarrow{AC}|}{|\overrightarrow{AB}|}$

- $\frac{|\overrightarrow{AB} \times \overrightarrow{AC}|}{|\overrightarrow{AB}|} = \frac{|(x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)|}{\sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}}$ (2)

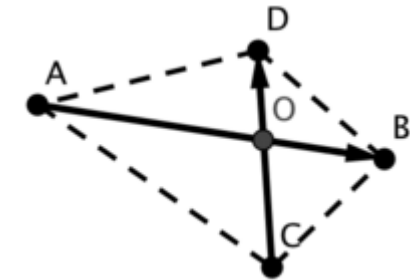
- From (1) and (2), if line l has the equation $ax + by + c = 0$ then the distance from point $P(x_P, y_P)$ to line l will be:

$$\text{dist}(l, P) = \frac{|ax_P + by_P + c|}{\sqrt{a^2 + b^2}}$$

Basic formular

- Two line segments AB and CD intersect when:
 - C and D are not on the same side of the line l_{AB} : $(\overrightarrow{AB} \times \overrightarrow{AC})(\overrightarrow{AB} \times \overrightarrow{AD}) \leq 0$
 - A and B are not on the same side of the line l_{CD} : $(\overrightarrow{CD} \times \overrightarrow{CA})(\overrightarrow{CD} \times \overrightarrow{CB}) \leq 0$
- Coordinates of intersection point $O(x_O, y_O)$ of 2 lines:
 - Equation of line l_{AB} : $a_1x + b_1y + c_1 = 0$
 - Equation of line l_{CD} : $a_2x + b_2y + c_2 = 0$

$$x_O = \frac{c_2b_1 - c_1b_2}{a_1b_2 - a_2b_1}$$
$$y_O = \frac{c_1a_2 - c_2a_1}{a_1b_2 - a_2b_1}$$



Basic formular

```
struct Point {
    int x, y;
    Point(int x, int y) : x(x), y(y) {}
};

struct Line{
    int a,b,c;
};

void makeLine(Point& A, Point& B, Line& L){
    L.a = A.y - B.y;    L.b = B.x - A.x;    L.c = A.x*B.y - A.y*B.x;
}

void intersection(Line& L1, Line& L2){
    double x = (L2.c*L1.b - L1.c*L2.b)*1.0/(L1.a*L2.b - L2.a*L1.b);
    double y = (L1.c*L2.a - L2.c*L1.a)*1.0/(L1.a*L2.b - L2.a*L1.b);
    cout << "Giao diem = (" << x << "," << y << ")" << endl;
}

int main(){
    Point A(3,1); Point B(6,4); Line LAB;
    Point C(2,5); Point D(7,0); Line LCD;
    makeLine(A,B,LAB); makeLine(C,D,LCD);    intersection(LAB,LCD);
}
```

Basic formular

A polygon is formed by a non-intersecting line with edges $P_0P_1, P_1P_2, P_2P_3, \dots, P_{n-1}P_0$

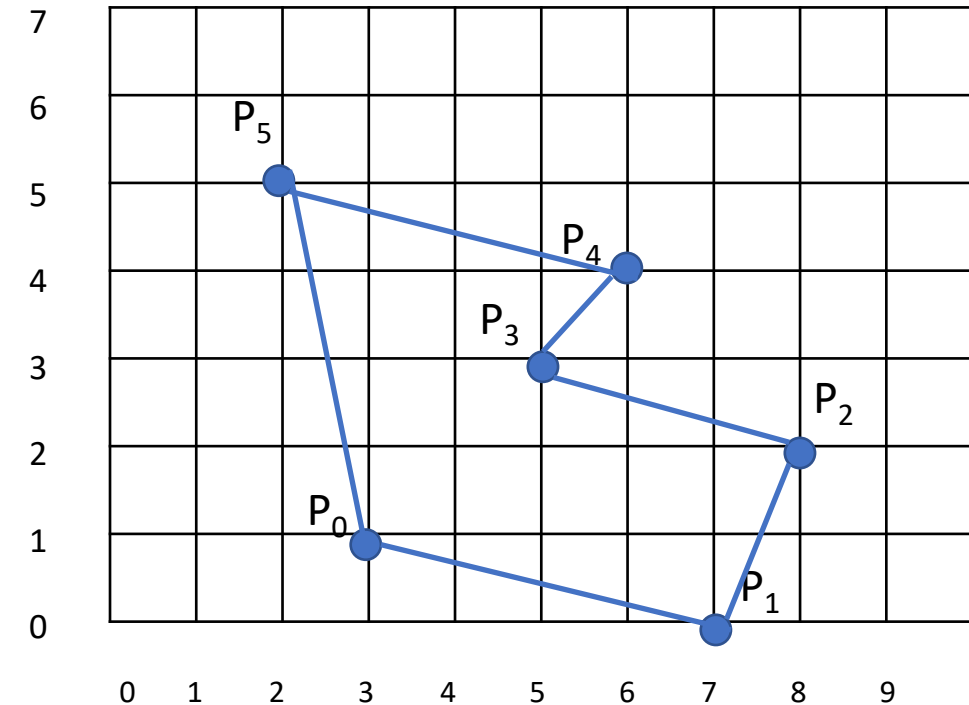
- In which vertex P_i has coordinates (x_i, y_i)

- Fix a vertex P_0

- Calculate the sum S :

- $S = \sum_{i=1}^{n-2} \overrightarrow{P_0P_i} \times \overrightarrow{P_0P_{i+1}}$

The area of the polygon is $\frac{|S|}{2}$



CONTENTS

- Basic formular
- **Find the convex hull (bao lồi)**
- Check if a point is inside a convex polygon (đa giác lồi)

Find convex hull (P.08.13.05)

Given a set of n points P_i ($i = 0, \dots, n-1$). Find the convex polygon with the smallest area containing all the given points.

- Data
 - Line 1: consists of a positive integer n ($3 \leq n \leq 100000$)
 - Line $i+1$ ($i = 1, 2, \dots, n$): consists of 2 integers x_i, y_i that are coordinate of the point P_i ($-1000 \leq x_i, y_i \leq 1000$)
- Result
 - Line 1: write the positive integer m as the number of points (vertices of the polygon) on the convex hull found
 - Line $i + 1$ ($i = 1, 2, \dots, m$): write 2 integers as the coordinates of the i^{th} point of the found convex hull

stdin	stdout
6	4
4 5	5 3
5 3	8 7
5 6	3 7
2 5	2 5
8 7	
3 7	

Find convex hull (P.08.13.05)

Given a set of n points P_i ($i = 0, \dots, n-1$). Find the convex polygon with the smallest area containing all the given points.

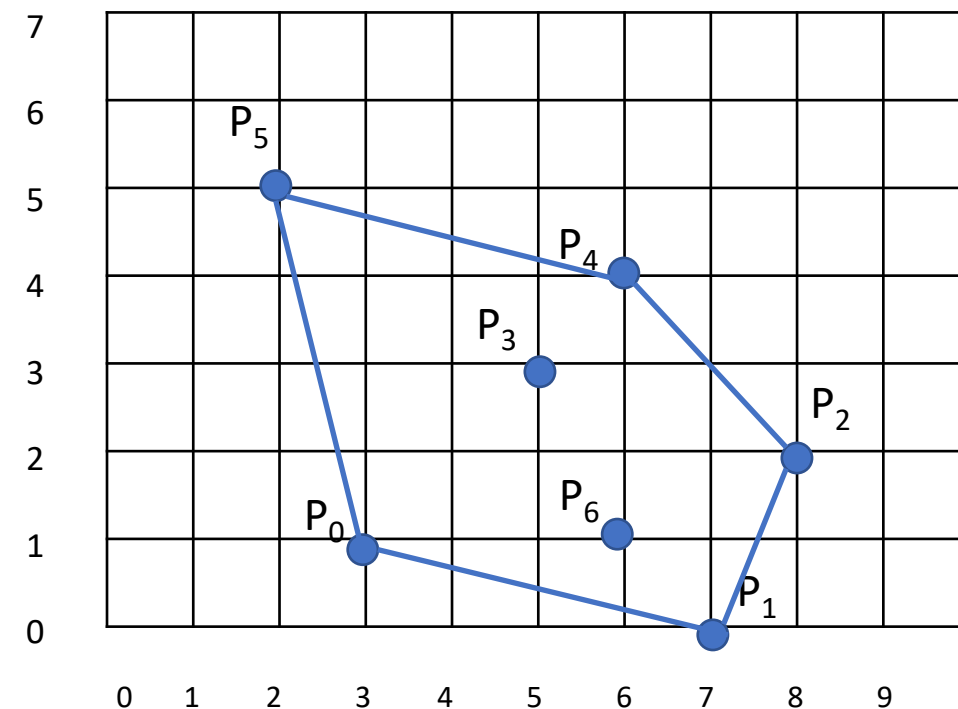
- **Graham Scan algorithm**

- Find the bottom left point - a point that definitely belongs to the convex hull. Make this point as point P_0
- Arrange the remaining $n - 1$ points at an angle with the origin as point P_0 .
- Create an empty stack S , add P_0 and P_1 to S
- For the remaining $n - 2$ points, repeat the following steps for each point P_i :
 - Repeatedly deleting the point at the top of stack S as long as the **CCW** of the following 3 points is not positive:
 - (a) The point (in the stack) is adjacent to the point at the top of the stack S
 - (b) Point at the top of stack S
 - (c) Point P_i
 - Add P_i to stack S
- At the end, we get the convex hull which is the vertices in clockwise order when taken from the stack S .

Find convex hull (P.08.13.05)

- The **counterclockwise** function determines the direction of rotation of ray OA to OB
- **CCW**(O, A, B) is defined as follows

$$\left\{ \begin{array}{l} 0, \text{ if } \overrightarrow{OA} \times \overrightarrow{OB} = 0 \\ -1, \text{ if } \overrightarrow{OA} \times \overrightarrow{OB} < 0 \\ +1, \text{ if } \overrightarrow{OA} \times \overrightarrow{OB} > 0 \end{array} \right.$$



Find convex hull (P.08.13.05) – Pseudo code

```
struct Point {
    int x, y;
};
Point P[N];
int n;
vector<Point> C;
void input(){
    read n;
    for(int i = 0; i < n; i++)
        read P[i].x, P[i].y;
}
```

```
dist2(Point a, Point b) {
    x = a.x - b.x;
    y = a.y - b.y;
    return x*x + y*y;
}
cross_product(Point O, Point A, Point B) {
    //tich vo huong 2 vector (O,A).(O,B)
    xa = A.x - O.x; ya = A.y - O.y;
    xb = B.x - O.x; yb = B.y - O.y;
    return xa * yb - ya * xb;
}
cmp(Point A, Point B){
    cp = cross_product(P[0],A,B);
    return cp == 0 ? dist2(P[0],A) < dist2(P[0],B) : cp > 0;
}
ccw(Point a, Point b, Point c) {
    cp = cross_product(a, b, c);
    return cp == 0 ? 0 : (cp < 0 ? -1 : 1);
}
```

Find convex hull (P.08.13.05) – Pseudo code

```
solve(){
    // find lowest point
    k = 0;
    for i = 1 to n - 1 do {
        if(P[i].y < P[k].y or P[i].y == P[k].y and P[i].x < P[k].x) k = i;
    }
    swap(P[0],P[k]); // let P[0] be the lowest point
    sort(P+1,P+n,cmp);
    C.push_back(P[0]);    C.push_back(P[1]);
    for i = 2 to n-1 do {
        while(C.size() > 1 and ccw(C[C.size()-2], C[C.size()-1],P[i]) <= 0)
            C.pop_back();
        C.push_back(P[i]);
    }
    print C[0], C[1], . . ., C[C.size-1];
}
```

```
main(){
    input();
    solve();
}
```

Find convex hull (P.08.13.05) – CODE

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5;
struct Point {
    int x, y;
    Point():x(0),y(0){}
    Point(int x, int y) : x(x), y(y) {}
};
Point P[N];
int n;
vector<Point> C;
void input(){
    cin >> n;
    for(int i = 0; i < n; i++)
        cin >> P[i].x >> P[i].y;
}
```

```
long long dist2(Point &a, Point &b) {
    long long x = a.x - b.x;
    long long y = a.y - b.y;
    return 1LL * x*x + 1LL * y*y;
}
long long cross_product(Point &O, Point &A, Point &B) {
    //tich vo huong 2 vector (O,A).(O,B)
    long long xa = A.x - O.x; long long ya = A.y - O.y;
    long long xb = B.x - O.x; long long yb = B.y - O.y;
    return 1LL * xa * yb - 1LL * ya * xb;
}
bool cmp(Point& A, Point& B){
    long long cp = cross_product(P[0],A,B);
    return cp == 0 ? dist2(P[0],A) < dist2(P[0],B) : cp > 0;
}
int ccw(Point &a, Point &b, Point &c) {
    long long cp = cross_product(a, b, c);
    return cp == 0 ? 0 : (cp < 0 ? -1 : 1);
}
```

Find convex hull (P.08.13.05) – CODE

```
void solve(){
    // find lowest point
    int k = 0;
    for(int i = 1; i < n; i++){
        if(P[i].y < P[k].y || P[i].y == P[k].y && P[i].x < P[k].x) k = i;
    }
    swap(P[0],P[k]); // let P[0] be the lowest point
    sort(P+1,P+n,cmp);
    C.push_back(P[0]);    C.push_back(P[1]);
    for(int i = 2; i < n; i++){
        while(C.size() > 1 && ccw(C[C.size()-2], C[C.size()-1],P[i]) <= 0)
            C.pop_back();
        C.push_back(P[i]);
    }
    for(int i = 0; i < C.size(); i++)
        cout << "(" << C[i].x << "," << C[i].y << ") ";
    cout << endl;
}
```

```
int main(){
    input();
    solve();
    return 0;
}
```

CONTENTS

- Basic formular
- Find the convex hull (bao lồi)
- **Check if a point is inside a convex polygon (đa giác lồi)**

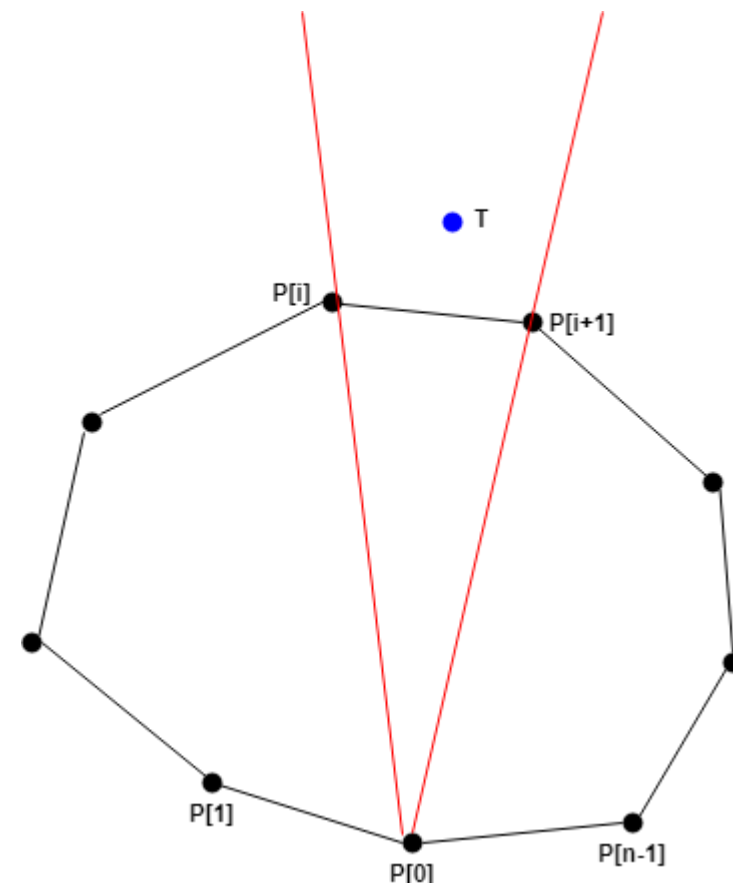
Check if a point is inside a convex polygon (P.08.13.06)

- Given n points $P_1, P_2, \dots, P_n$ on the plane, integer coordinates. Give K points $T_1, T_2, \dots, T_k$. Check whether the point set $P_1, P_2, \dots, P_n$ performs a convex hull or not? If yes, check whether each point $T_1, T_2, \dots, T_k$ is inside (or on the edge) of that convex polygon or not? (points all have integer coordinates ranging from -1000 to 1000)
- Data
 - Line 1: consists of a positive integer n ($3 \leq n \leq 10000$)
 - Line $i+1$ ($i = 1, 2, \dots, n$): consists of 2 integers x, y that are coordinate of the point P_i
 - Line $n+2$: consists of a positive integer K ($1 \leq K \leq 100000$)
 - Line $k + n + 2$ ($k = 1, 2, \dots, K$): consists of 2 integers x, y that are coordinate of the point T_k
- Result
 - Line k^{th} : write 1 if set of points $P_1, P_2, \dots, P_n$ forms a convex polygon and point T_k lies inside or on the edge of that polygon; and write out 0, otherwise

stdin	stdout
4	0
5 6	1
3 7	1
2 5	0
5 3	
4	
8 7	
4 5	
3 7	
0 0	

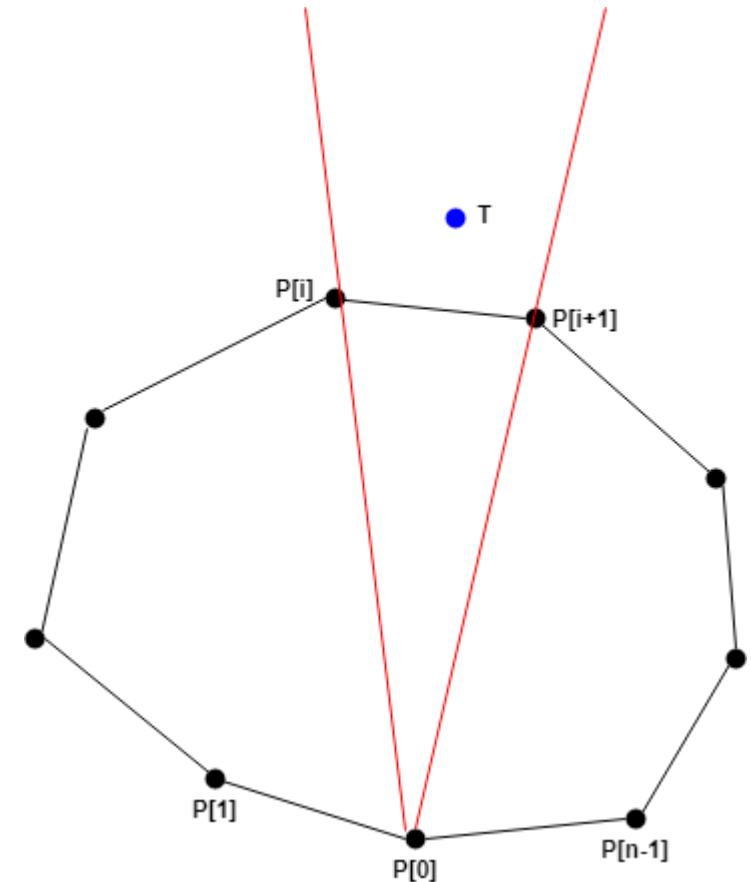
Check if a point is inside a convex polygon (P.08.13.06)

- Given a convex polygon n point $P[i]$, check whether point T is inside the given convex polygon or not?
- Check if point T is on the same side as point $P[n-1]$ compared to the line passing through $P[0]$ and $P[1]$.
- Check if point T is on the same side as point $P[1]$ compared to the line passing through $P[0]$ and $P[n-1]$.
- Use binary search to find point $P[i]$ that satisfies point $P[i+1]$ on the other side of point $P[1]$ compared to the line passing through $P[0]$ and T , and point $P[i]$ lies on the same side as point $P[1]$ compared to the line passing through $P[0]$ and T .
- T is inside the polygon when $P[0]$ and T are on the same side of the line passing through $P[i]$ and $P[i+1]$.



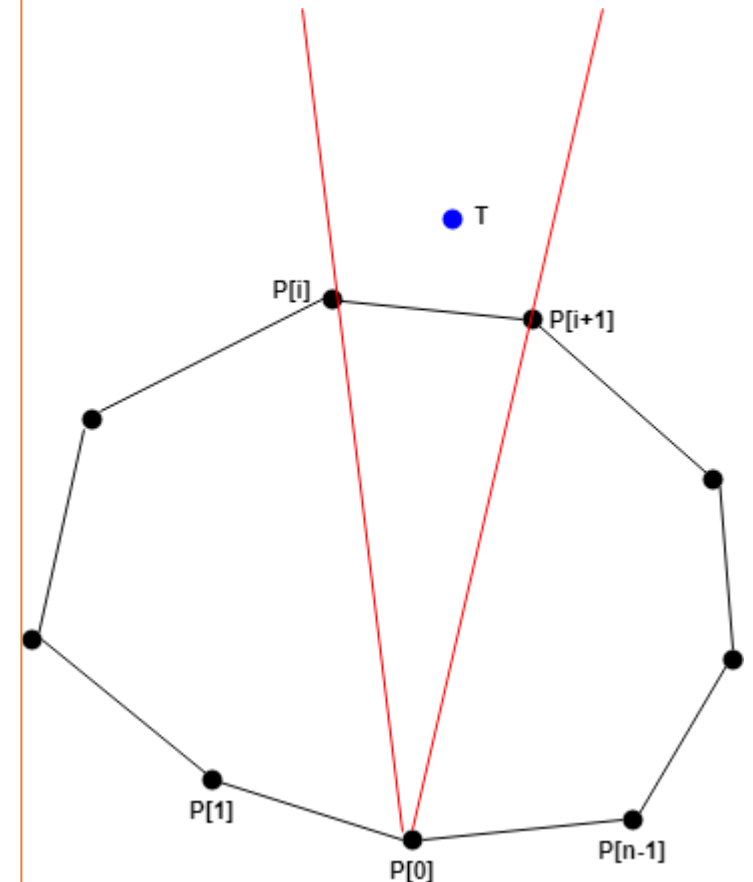
Check if a point is inside a convex polygon (P.08.13.06) – Pseudo code

```
ccw(Point a, Point b, Point c) {  
    cp = cross_product(a, b, c);  
    return cp == 0 ? 0 : (cp < 0 ? -1 : 1);  
}  
  
same_side(Point a, Point b, Point c, Point d) {  
    // return true if c and d are in the same side of the line (a,b)  
    sc = ccw(a, b, c);  
    sd = ccw(a, b, d);  
    return sc * sd >= 0;  
}
```



Check if a point is inside a convex polygon (P.08.13.06) – Pseudo code

```
checkInsideConvexHull(vector<Point> C, Point q) {  
    last = C.size() - 1;  
    if (same_side(C[0], C[1], C[last], q) and // C[last] and q are on the same  
                                                // side of C[0]C[1]  
        same_side(C[0], C[last], C[1], q)) { // C[1] and q are on different side  
                                                // of C[last]C[0]  
  
        l = 1;    r = C.size() - 1;  
        while (r - l > 1) do {  
            mid = (l + r) / 2;  
            if (same_side(C[0], C[mid], C[last], q)) l = mid; else r = mid;  
        }  
        if ( not same_side(C[1], C[r], C[0], q))  
            return false;  
        else return true;  
    }  
    return false;  
}
```



Check if a point is inside a convex polygon (P.08.13.06) – CODE

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5+1;
struct Point {
    int x, y;
    Point():x(0),y(0){}
    Point(int x, int y) : x(x), y(y) {}
};
Point P[N];
int n;
vector<Point> C;
Point O; // goc
long long dist2(Point &a, Point &b) {
    long long x = a.x - b.x;
    long long y = a.y - b.y;
    return 1LL * x*x + 1LL * y*y;
}
```

```
long long cross_product(Point &O, Point &A, Point &B) {
    //tich co huong 2 vector (O,A) x (O,B)
    long long xa = A.x - O.x; long long ya = A.y - O.y;
    long long xb = B.x - O.x; long long yb = B.y - O.y;
    return 1LL * xa * yb - 1LL * ya * xb;
}
bool cmp(Point& A, Point& B){
    long long cp = cross_product(O,A,B);
    return cp == 0 ? dist2(O,A) < dist2(O,B) : cp > 0;
}
int ccw(Point &a, Point &b, Point &c) {
    long long cp = cross_product(a, b, c);
    return cp == 0 ? 0 : (cp < 0 ? -1 : 1);
}
bool same_side(Point &a, Point &b, Point &c, Point &d) {
    // return true if c and d are in the same side of the line (a,b)
    int sc = ccw(a, b, c);
    int sd = ccw(a, b, d);
    return sc * sd >= 0;
}
```

Check if a point is inside a convex polygon (P.08.13.06) – CODE

```
void computeConvexHull(Point* P, int n){
    C.clear(); int k = 0;
    for(int i = 1; i < n; i++){
        if(P[i].y < P[k].y ||
            P[i].y == P[k].y && P[i].x < P[k].x) k = i;
    }
    swap(P[0],P[k]); // let P[0] be the lowest point
    O = P[0]; // update goc
    sort(P+1,P+n,cmp);
    C.push_back(P[0]);    C.push_back(P[1]);
    for(int i = 2; i < n; i++){
        while(C.size() > 1
            && ccw(C[C.size()-2], C[C.size()-1],P[i]) <= 0)
            C.pop_back();
        C.push_back(P[i]);
    }
}
```

```
int checkInSideConvexHull(vector<Point> P, Point& T) {
    int last = P.size() - 1;
    if (same_side(P[0], P[1], P[last], T) &&
        same_side(P[0], P[last], P[1], T)) {
        int l = 1;    int r = P.size() - 1;
        while (r - l > 1) {
            int mid = (l + r) >> 1;
            if (same_side(P[0], P[mid], P[last], T)) l = mid;
            else r = mid;
        }
        if (!same_side(P[1], P[r], P[0], T)) return 0;
        else return 1;
    }
    return 0;
}
```

Check if a point is inside a convex polygon (P.08.13.06) – CODE

```
int main(){
    scanf("%d",&n);
    for(int i = 0; i < n; i++){
        scanf("%d%d",&(P[i].x),&(P[i].y));
    }
    computeConvexHull(P,n);
    int res = 1;
    if(C.size() != n) res = 0;
    int K;
    scanf("%d",&K);
    for(int k = 1; k <= K; k++){
        int x,y;        scanf("%d%d",&x,&y);
        Point p(x,y);
        if(res == 1)
            res = checkInSideConvexHull(C,p);
        printf("%d\n",res);
    }
    return 0;
}
```

A graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern of red dots. The dots are arranged in concentric, slightly irregular rings, creating a sense of depth and movement. In the center of this pattern, the word "HUST" is written in a bold, white, sans-serif font.

HUST

THANK YOU !